GUARD APP TESTING STACK PROOF OF CONCEPT

INTRODUCTION

The aim of this task was to explore and evaluate an automated testing strategy for the SecureShift Guard App, which is developed using React Native. The main objective was to find a testing approach that is practical, easy to maintain, and can work smoothly across both iOS and Android platforms.

To achieve this, I researched different testing tools and approaches, and then built a small proof of concept (PoC) to see how well the chosen tools would perform in the actual project environment.

TESTING IN REACT NATIVE

From my research, testing in React Native is generally split into three main types. These include unit testing, which focuses on small parts of the code like functions or logic; component testing, which checks UI elements and how users interact with them; and end-to-end (E2E) testing, which looks at complete user flows such as logging in or navigating through the app.

In most real-world applications, developers don't rely on just one type of testing. Instead, they use a combination of all three to make sure both the functionality and overall user experience are properly covered.

TOOLS I LOOKED AT

Before deciding on the final tools, I looked into a few commonly used options.

- Jest: Jest is one of the most widely used testing frameworks for React Native. It's quite fast and usually comes pre-configured in many projects, which makes it really easy to start using without much setup.
- React Native Testing Library: It is often used alongside Jest. It focuses more on testing from the user's perspective rather than the internal code. Basically, it checks what the user actually sees on the screen and how they interact with it.
- Detox: For end-to-end testing, I explored Detox. It's capable of testing complete user flows, like logging in or navigating through the app. However, it seemed a bit more complex and required more setup compared to the other tools.

- Maestro: I also looked at Maestro, which is another tool for end-to-end testing. Compared to Detox, it seemed simpler and easier to work with. It also supports both Android and iOS, which makes it a good option to consider for future use.

Selected Testing Approach

After going through the different options, I decided to use Jest and React Native Testing Library.

I chose these mainly because they are easy to set up, work well with Expo, and are quite effective for testing UI behaviour. They also don't make the project overly complicated, which is important when it comes to maintaining the code in the long run.

I didn't fully implement end-to-end testing as part of this task. However, based on what I found during my research, Maestro seems like a good option to explore next if we want to test complete app flows in the future.

Proof of Concept

For the proof of concept, I created a test for the Login screen since it is one of the main features of the application and includes both UI elements and user interaction.

The test checks a few key things, including:

- whether the screen renders properly
- whether the email and password input fields are visible
- whether the login button is present
- basic interaction, such as pressing the login button

To implement this, I used Jest along with React Native Testing Library. The render function was used to load the Login screen component in a test environment, and queries like 'getByText' and 'getByPlaceholderText' were used to make sure the elements appear correctly on the screen.

I also used 'fireEvent' to simulate a user pressing the login button. This helps confirm that the component responds properly to user actions, rather than just checking if the UI is displayed.

Even though the test is fairly simple, it still shows how automated testing can be used to check both the structure and behaviour of a screen. It also gives a good starting point for expanding into more advanced tests, such as form validation, API calls, and navigation between screens.

IMPLEMENTATION

To set up the testing environment, I first configured Jest using the Expo preset (jest-expo). This makes it possible to run tests in a React Native environment without needing an actual device or emulator.

I then created a '__tests__' folder to organise the test files, following the usual Jest structure. Inside this folder, I added a test file specifically for the Login screen.

Using React Native Testing Library, I rendered the Login screen component and used queries like 'getByText' and 'getByPlaceholderText' to check if the UI elements were showing up correctly. These queries are useful because they mimic how a user would actually see and interact with the app, rather than focusing on the internal code.

I also used 'fireEvent' to simulate user interaction, such as pressing the login button. This helped confirm that the component responds properly when a user interacts with it.

While setting everything up, I ran into an issue with 'AsyncStorage', since it's a native module and isn't available in the Jest testing environment. To fix this, I created a Jest setup file and mocked 'AsyncStorage'. After doing this, the tests were able to run without any problems.

Overall, once the initial setup was done, the implementation was quite straightforward. It provided a working example of how automated testing can be added to the project in a practical way. This setup can be reused for other screens in the application by following the same structure, making it easy to expand test coverage over time.

CHALLENGES

During the implementation, I ran into a few issues while setting up the testing environment.

The main problem was related to 'AsyncStorage'. When I first tried running the tests, they failed because 'AsyncStorage' is a native module and isn't available in the Jest testing environment. Because of this, the test suite was crashing.

To fix the issue, I created a Jest setup file and mocked 'AsyncStorage' using the official documentation as a reference. After setting this up, the tests were able to run properly without relying on native functionality.

Another smaller issue I faced was making sure the test queries matched the actual UI elements. For example, one of the tests failed because the placeholder text I used didn't exactly match what was written in the Login screen. I had to go back, check the component carefully, and update the test so that it matched correctly.

Overall, these challenges helped me better understand the difference between a testing environment and the real app environment. It also gave me a clearer idea of how mocking is used to handle dependencies when writing tests.

CONCLUSION

This task gave me a good understanding of how automated testing can be introduced into a React Native project in a practical way.

By setting up Jest and React Native Testing Library, I was able to create a simple but functional testing environment and apply it to a real feature in the app. The proof of concept showed that both the UI and basic user interactions can be tested without needing a physical device.

Even though the current setup is quite basic, it still acts as a solid starting point that can be built on in the future. For example, more tests can be added for other screens, and tools like Maestro could be explored to test complete user flows across the app.

In general, this approach is easy to maintain, works well with the existing Expo setup, and can scale as the project grows. It also helps increase confidence in the app by catching issues earlier during development.

EVIDENCE

1. Successful execution of automated tests using Jest

```
PS C:\SecureShift1\SecureShift\guard_app> npm test

> secureshift-guardapp@1.0.0 test
> jest

[baseline-browser-mapping] The data in this module is over two months old.  To ensure accurate Baseline data, please update: `npm i baseline-browser-mapping@latest -D`
  console.error
    An update to Icon inside a test was not wrapped in act(...).

    When testing, code that causes React state updates should be wrapped into act(...):

    act(() => {
      /* fire events that update state */
    });
    /* assert on the output */

    This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

      at node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11801:19
      at runWithFiberInDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:1574:13)
      at warnIfUpdatesNotWrappedWithActDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11800:9)
      at scheduleUpdateOnFiber (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:10326:9)
      at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)
      at Icon.Object.<anonymous>.process.env.NODE_ENV.Component.setState (node_modules/react/cjs/react.development.js:632:20)
      at Icon.setState (node_modules/@expo/vector-icons/src/createIconSet.tsx:181:31)
      at asyncGeneratorStep (node_modules/@babel/runtime/helpers/asyncToGenerator.js:3:17)
      at _next (node_modules/@babel/runtime/helpers/asyncToGenerator.js:17:9)

 PASS  __tests__/Login.test.js
  Login Screen
    √ renders the login form correctly (354 ms)
    √ shows validation error when fields are empty (280 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.433 s, estimated 8 s
Ran all test suites.
PS C:\SecureShift1\SecureShift\guard_app>
```

2. Login screen test implementation using React Native Testing Library

.env | SettingsScreen.tsx | AppNavigator.tsx | ReleaseNotesScreen.tsx | package.json M | Login.test.js U | jest.setup.js U | loginscreen.tsx

guard_app > __tests__ > Login.test.js > ...

```
import React from 'react';
import { render, fireEvent } from '@testing-library/react-native';
import LoginScreen from '../src/screen/loginscreen';

describe('Login Screen', () => {
  const navigation = {
    navigate: jest.fn(),
    reset: jest.fn(),
  };

  it('renders the login form correctly', () => {
    const { getByPlaceholderText, getByText } = render(
      <LoginScreen navigation={navigation} />
    );

    expect(getByText('Login with your email and password')).toBeTruthy();
    expect(getByPlaceholderText('Enter your email')).toBeTruthy();
    expect(getByPlaceholderText('Enter your password')).toBeTruthy();
    expect(getByText('Login')).toBeTruthy();
  });

  it('shows validation error when fields are empty', () => {
    const { getByText } = render(
      <LoginScreen navigation={navigation} />
    );

    const loginButton = getByText('Login');

    fireEvent.press(loginButton);

    // We expect validation to trigger
    expect(getByText('Login')).toBeTruthy(); // button still there
  });
});
```

3. Jest setup file with AsyncStorage mocking

.env | SettingsScreen.tsx | AppNavigator.tsx | ReleaseNotesScreen.tsx | package.json M | Login.test.js U | jest.setup.js U | loginscreen.tsx

guard_app > jest.setup.js > ...

```
import mockAsyncStorage from '@react-native-async-storage/async-storage/jest/async-storage-mock';

jest.mock('@react-native-async-storage/async-storage', () => mockAsyncStorage);
```